

Soluções Revisadas – Programas, Máquinas e Computação

Versão revisada, com notação mais formal e soluções colocadas abaixo de cada enunciado.

- Critérios adotados nesta versão:
- Para máquina genérica, foram usadas leitura, escrita, atribuições e testes usuais.
- Para a máquina 2_REG, foram usadas apenas as operações da máquina: a_zero, subtrair_a e adicionar_b.
- Nos programas recursivos, a operação vazia $\surd$ representa o encerramento natural do ramo.
- Nos programas iterativos da 2_REG, a escrita foi mantida o mais próxima possível do formalismo do livro.

8) Fatorial de n – máquina genérica

Enunciado: Desenvolver um programa sobre máquina genérica que calcule o fatorial de n, com mensagem de erro para entrada inválida.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Faça ler(n) vá_para R2;  
R2: Se n <= 0 então vá_para R3 senão vá_para R4;  
R3: Faça escrever(erro) vá_para Rx;  
R4: Faça f = 1 vá_para R5;  
R5: Faça i = 1 vá_para R6;  
R6: Se i <= n então vá_para R7 senão vá_para R10;  
R7: Faça f = f * i vá_para R8;  
R8: Faça i = i + 1 vá_para R6;  
R10: Faça escrever(f) vá_para Rx;
```

Solução – Programa iterativo:

```
(ler(n);  
  se n <= 0 então escrever(erro)  
  senão  
    (f = 1; i = 1;  
     enquanto i <= n faça  
       (f = f * i; i = i + 1);  
     escrever(f)))
```

Solução – Programa recursivo:

P é ler(n); (se n <= 0 então escrever(erro) senão f = 1; i = 1; R) onde
R def (se i <= n então f = f * i; i = i + 1; R senão escrever(f))

9) Constante e – máquina genérica

Enunciado: Estimar $e = 1/0! + 1/1! + \dots + 1/n!$, com mensagem de erro se n for inválido.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Faça ler(n) vá_para R2;
R2: Se n <= 0 então vá_para R3 senão vá_para R4;
R3: Faça escrever(erro) vá_para Rx;
R4: Faça f = 1 vá_para R5;
R5: Faça e = 1 vá_para R6;
R6: Faça i = 1 vá_para R7;
R7: Se i <= n então vá_para R8 senão vá_para R11;
R8: Faça f = f * i vá_para R9;
R9: Faça e = e + 1/f vá_para R10;
R10: Faça i = i + 1 vá_para R7;
R11: Faça escrever(e) vá_para Rx;
```

Solução – Programa iterativo:

```
(ler(n);
se n <= 0 então escrever(erro)
senão
  (f = 1; e = 1; i = 1;
  enquanto i <= n faça
    (f = f * i; e = e + 1/f; i = i + 1);
  escrever(e)))
```

Solução – Programa recursivo:

```
P é ler(n); (se n <= 0 então escrever(erro) senão f = 1; e = 1; i = 1; R)
onde
R def (se i <= n então f = f * i; e = e + 1/f; i = i + 1; R senão
escrever(e))
```

10) Série de e^x – máquina genérica

Enunciado: Calcular $e^x = x^0/0! + x^1/1! + \dots + x^n/n!$, com n inteiro positivo e x real qualquer.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Faça ler(n) vá_para R2;
R2: Faça ler(x) vá_para R3;
R3: Se n <= 0 então vá_para R4 senão vá_para R5;
R4: Faça escrever(erro) vá_para Rx;
R5: Faça s = 1 vá_para R6;
R6: Faça p = 1 vá_para R7;
R7: Faça f = 1 vá_para R8;
R8: Faça i = 1 vá_para R9;
R9: Se i <= n então vá_para R10 senão vá_para R14;
R10: Faça p = p * x vá_para R11;
R11: Faça f = f * i vá_para R12;
R12: Faça s = s + p/f vá_para R13;
R13: Faça i = i + 1 vá_para R9;
R14: Faça escrever(s) vá_para Rx;
```

Solução – Programa iterativo:

```
(ler(n); ler(x);
se n <= 0 então escrever(erro)
senão
  (s = 1; p = 1; f = 1; i = 1;
  enquanto i <= n faça
    (p = p * x; f = f * i; s = s + p/f; i = i + 1);
  escrever(s)))
```

Solução – Programa recursivo:

```
P é ler(n); ler(x); (se n <= 0 então escrever(erro) senão s = 1; p = 1; f = 1; i = 1; R) onde
R def (se i <= n então p = p * x; f = f * i; s = s + p/f; i = i + 1; R
senão escrever(s))
```

11) Série de Fibonacci – máquina genérica

Enunciado: Apresentar os n primeiros termos da série de Fibonacci, com mensagem de erro se n for inválido.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Faça ler(n) vá_para R2;
R2: Se n <= 0 então vá_para R3 senão vá_para R4;
R3: Faça escrever(erro) vá_para Rx;
R4: Faça escrever(1) vá_para R5;
R5: Se n = 1 então vá_para Rx senão vá_para R6;
R6: Faça escrever(1) vá_para R7;
R7: Faça a = 1 vá_para R8;
R8: Faça b = 1 vá_para R9;
R9: Faça i = 3 vá_para R10;
R10: Se i <= n então vá_para R11 senão vá_para Rx;
R11: Faça c = a + b vá_para R12;
R12: Faça escrever(c) vá_para R13;
R13: Faça a = b vá_para R14;
R14: Faça b = c vá_para R15;
R15: Faça i = i + 1 vá_para R10;
```

Solução – Programa iterativo:

```
(ler(n);
  se n <= 0 então escrever(erro)
  senão
    (escrever(1);
      se n = 1 então √
      senão
        (escrever(1); a = 1; b = 1; i = 3;
          enquanto i <= n faça
            (c = a + b; escrever(c); a = b; b = c; i = i + 1))))
```

Solução – Programa recursivo:

```
P é ler(n); (se n <= 0 então escrever(erro) senão escrever(1); S) onde
S def (se n = 1 então √ senão escrever(1); a = 1; b = 1; i = 3; R)
R def (se i <= n então c = a + b; escrever(c); a = b; b = c; i = i + 1; R
senão √)
```

14) Série $S = 1 + 1/2 + 1/4 + 1/6 + \dots$ – máquina genérica

Enunciado: Calcular a soma dos n termos da série, com mensagem de erro se n for inválido.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Faça ler(n) vá_para R2;
R2: Se n <= 0 então vá_para R3 senão vá_para R4;
R3: Faça escrever(erro) vá_para Rx;
R4: Faça s = 1 vá_para R5;
R5: Faça d = 2 vá_para R6;
R6: Faça k = 2 vá_para R7;
R7: Se k <= n então vá_para R8 senão vá_para R11;
R8: Faça s = s + 1/d vá_para R9;
R9: Faça d = d + 2 vá_para R10;
R10: Faça k = k + 1 vá_para R7;
R11: Faça escrever(s) vá_para Rx;
```

Solução – Programa iterativo:

```
(ler(n);
se n <= 0 então escrever(erro)
senão
  (s = 1; d = 2; k = 2;
  enquanto k <= n faça
    (s = s + 1/d; d = d + 2; k = k + 1);
  escrever(s)))
```

Solução – Programa recursivo:

```
P é ler(n); (se n <= 0 então escrever(erro) senão s = 1; d = 2; k = 2; R)
onde
R def (se k <= n então s = s + 1/d; d = d + 2; k = k + 1; R senão
escrever(s))
```

18) Somatória dos termos da série de Fibonacci – máquina genérica

Enunciado: Calcular a soma dos n primeiros termos de Fibonacci, com mensagem de erro se n for inválido.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Faça ler(n) vá_para R2;
R2: Se n <= 0 então vá_para R3 senão vá_para R4;
R3: Faça escrever(erro) vá_para Rx;
R4: Faça s = 1 vá_para R5;
R5: Se n = 1 então vá_para R16 senão vá_para R6;
R6: Faça a = 1 vá_para R7;
R7: Faça b = 1 vá_para R8;
R8: Faça s = 2 vá_para R9;
R9: Faça i = 3 vá_para R10;
R10: Se i <= n então vá_para R11 senão vá_para R16;
R11: Faça c = a + b vá_para R12;
R12: Faça s = s + c vá_para R13;
R13: Faça a = b vá_para R14;
R14: Faça b = c vá_para R15;
R15: Faça i = i + 1 vá_para R10;
R16: Faça escrever(s) vá_para Rx;
```

Solução – Programa iterativo:

```
(ler(n);
se n <= 0 então escrever(erro)
senão
  (s = 1;
  se n = 1 então escrever(s)
  senão
    (a = 1; b = 1; s = 2; i = 3;
    enquanto i <= n faça
      (c = a + b; s = s + c; a = b; b = c; i = i + 1);
    escrever(s))))
```

Solução – Programa recursivo:

```
P é ler(n); (se n <= 0 então escrever(erro) senão s = 1; S) onde
S def (se n = 1 então escrever(s) senão a = 1; b = 1; s = 2; i = 3; R)
R def (se i <= n então c = a + b; s = s + c; a = b; b = c; i = i + 1; R
senão escrever(s))
```

19) Somatória de $2k$, $k = 1..n$ – máquina genérica

Enunciado: Calcular $2 \cdot 1 + 2 \cdot 2 + \dots + 2 \cdot n$, com mensagem de erro se n for inválido.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Faça ler(n) vá_para R2;
R2: Se n <= 0 então vá_para R3 senão vá_para R4;
R3: Faça escrever(erro) vá_para Rx;
R4: Faça s = 0 vá_para R5;
R5: Faça k = 1 vá_para R6;
R6: Se k <= n então vá_para R7 senão vá_para R10;
R7: Faça s = s + 2*k vá_para R8;
R8: Faça k = k + 1 vá_para R6;
R10: Faça escrever(s) vá_para Rx;
```

Solução – Programa iterativo:

```
(ler(n);
se n <= 0 então escrever(erro)
senão
  (s = 0; k = 1;
  enquanto k <= n faça
    (s = s + 2*k; k = k + 1);
  escrever(s)))
```

Solução – Programa recursivo:

P é ler(n); (se n <= 0 então escrever(erro) senão s = 0; k = 1; R) onde
R def (se k <= n então s = s + 2*k; k = k + 1; R senão escrever(s))

22) 2_REG – B = A

Enunciado: Implementar a função $B = A$ na máquina 2_REG.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Se a_zero então vá_para Rx senão vá_para R2;  
R2: Faça subtrair_a vá_para R3;  
R3: Faça adicionar_b vá_para R1;
```

Solução – Programa iterativo:

```
(até a_zero faça (subtrair_a; adicionar_b))
```

Solução – Programa recursivo:

```
P é R onde  
R def (se a_zero então √ senão subtrair_a; adicionar_b; R)
```

Observações: Cada unidade retirada de A é imediatamente adicionada a B. Ao final, $A = 0$ e B recebe exatamente o valor inicial de A.

23) 2_REG – $B = A + 2$

Enunciado: Implementar a função $B = A + 2$ na máquina 2_REG.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Se a_zero então vá_para R4 senão vá_para R2;  
R2: Faça subtrair_a vá_para R3;  
R3: Faça adicionar_b vá_para R1;  
R4: Faça adicionar_b vá_para R5;  
R5: Faça adicionar_b vá_para Rx;
```

Solução – Programa iterativo:

```
((até a_zero faça (subtrair_a; adicionar_b));  
adicionar_b;  
adicionar_b)
```

Solução – Programa recursivo:

```
P é R onde  
R def (se a_zero então adicionar_b; adicionar_b senão subtrair_a;  
adicionar_b; R)
```

Observações: Primeiro copia A para B; depois acrescenta mais 2 unidades a B.

24) 2_REG – B = A – 2

Enunciado: Implementar a função $B = A - 2$ na máquina 2_REG, adotando resultado 0 quando $A < 2$.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Se a_zero então vá_para Rx senão vá_para R2;
R2: Faça subtrair_a vá_para R3;
R3: Se a_zero então vá_para Rx senão vá_para R4;
R4: Faça subtrair_a vá_para R5;
R5: Se a_zero então vá_para Rx senão vá_para R6;
R6: Faça subtrair_a vá_para R7;
R7: Faça adicionar_b vá_para R5;
```

Solução – Programa iterativo:

```
(se a_zero então √
senão
  (subtrair_a;
   se a_zero então √
   senão
     (subtrair_a;
      até a_zero faça (subtrair_a; adicionar_b))))
```

Solução – Programa recursivo:

```
P é (se a_zero então √ senão subtrair_a; S) onde
S def (se a_zero então √ senão subtrair_a; R)
R def (se a_zero então √ senão subtrair_a; adicionar_b; R)
```

Observações: As duas primeiras unidades de A são descartadas; a partir daí, cada nova unidade removida de A gera uma unidade em B.

25) 2_REG – $B = A * 3$

Enunciado: Implementar a função $B = A * 3$ na máquina 2_REG.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Se a_zero então vá_para Rx senão vá_para R2;  
R2: Faça subtrair_a vá_para R3;  
R3: Faça adicionar_b vá_para R4;  
R4: Faça adicionar_b vá_para R5;  
R5: Faça adicionar_b vá_para R1;
```

Solução – Programa iterativo:

```
(até a_zero faça (subtrair_a; adicionar_b; adicionar_b; adicionar_b))
```

Solução – Programa recursivo:

```
P é R onde  
R def (se a_zero então √ senão subtrair_a; adicionar_b; adicionar_b;  
adicionar_b; R)
```

Observações: Cada unidade retirada de A gera três adições em B.

26) 2_REG – B = A / 3

Enunciado: Implementar a função $B = \lfloor A/3 \rfloor$ na máquina 2_REG.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Se a_zero então vá_para Rx senão vá_para R2;  
R2: Faça subtrair_a vá_para R3;  
R3: Se a_zero então vá_para Rx senão vá_para R4;  
R4: Faça subtrair_a vá_para R5;  
R5: Se a_zero então vá_para Rx senão vá_para R6;  
R6: Faça subtrair_a vá_para R7;  
R7: Faça adicionar_b vá_para R1;
```

Solução – Programa iterativo:

Padrão iterativo formal:

```
(se a_zero então √  
  senão  
    (subtrair_a;  
      se a_zero então √  
        senão  
          (subtrair_a;  
            se a_zero então √  
              senão  
                (subtrair_a;  
                  adicionar_b;  
                    repetir o mesmo padrão))))
```

Solução – Programa recursivo:

```
P é R onde  
R def (se a_zero então √ senão subtrair_a; S)  
S def (se a_zero então √ senão subtrair_a; T)  
T def (se a_zero então √ senão subtrair_a; adicionar_b; R)
```

Observações: O programa só adiciona 1 a B quando consegue retirar um bloco completo de 3 unidades de A.

27) 2_REG – $B = (A / 2) * 3$

Enunciado: Implementar a função $B = 3 \cdot \lfloor A/2 \rfloor$ na máquina 2_REG.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Se a_zero então vá_para Rx senão vá_para R2;  
R2: Faça subtrair_a vá_para R3;  
R3: Se a_zero então vá_para Rx senão vá_para R4;  
R4: Faça subtrair_a vá_para R5;  
R5: Faça adicionar_b vá_para R6;  
R6: Faça adicionar_b vá_para R7;  
R7: Faça adicionar_b vá_para R1;
```

Solução – Programa iterativo:

Padrão iterativo formal:

```
(se a_zero então √  
senão  
  (subtrair_a;  
   se a_zero então √  
   senão  
     (subtrair_a;  
      adicionar_b; adicionar_b; adicionar_b;  
      repetir o mesmo padrão)))
```

Solução – Programa recursivo:

```
P é R onde  
R def (se a_zero então √ senão subtrair_a; S)  
S def (se a_zero então √ senão subtrair_a; adicionar_b; adicionar_b;  
adicionar_b; R)
```

Observações: O programa exige pares completos em A; para cada par, adiciona 3 unidades em B.

28) 2_REG – $B = A + (A \% 3)$

Enunciado: Implementar a função $B = A + (A \bmod 3)$ na máquina 2_REG.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Se a_zero então vá_para Rx senão vá_para R2;
R2: Faça subtrair_a vá_para R3;
R3: Se a_zero então vá_para R10 senão vá_para R4;
R4: Faça subtrair_a vá_para R5;
R5: Se a_zero então vá_para R20 senão vá_para R6;
R6: Faça subtrair_a vá_para R7;
R7: Faça adicionar_b vá_para R8;
R8: Faça adicionar_b vá_para R9;
R9: Faça adicionar_b vá_para R1;
R10: Faça adicionar_b vá_para R11;
R11: Faça adicionar_b vá_para Rx;
R20: Faça adicionar_b vá_para R21;
R21: Faça adicionar_b vá_para R22;
R22: Faça adicionar_b vá_para R23;
R23: Faça adicionar_b vá_para Rx;
```

Solução – Programa iterativo:

Padrão iterativo formal:
tratar blocos completos de 3 unidades de A com 3 adições em B;
ao final, se sobrar 1 unidade, adicionar 2 em B;
se sobrar 2 unidades, adicionar 4 em B;
se não sobrar nenhuma unidade, encerrar.

Solução – Programa recursivo:

```
P é R onde
R def (se a_zero então √ senão subtrair_a; S)
S def (se a_zero então adicionar_b; adicionar_b senão subtrair_a; T)
T def (se a_zero então adicionar_b; adicionar_b; adicionar_b; adicionar_b
senão subtrair_a; adicionar_b; adicionar_b; adicionar_b; R)
```

Observações: Cada bloco completo de 3 unidades contribui com 3 para B; os restos 1 e 2 são tratados separadamente como +2 e +4.

29) 2_REG – $B = (A / 2) - 3$

Enunciado: Implementar a função $B = \lfloor A/2 \rfloor - 3$ na máquina 2_REG, adotando resultado 0 quando $\lfloor A/2 \rfloor < 3$.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Se a_zero então vá_para Rx senão vá_para R2;
R2: Faça subtrair_a vá_para R3;
R3: Se a_zero então vá_para Rx senão vá_para R4;
R4: Faça subtrair_a vá_para R5;
R5: Se a_zero então vá_para Rx senão vá_para R6;
R6: Faça subtrair_a vá_para R7;
R7: Se a_zero então vá_para Rx senão vá_para R8;
R8: Faça subtrair_a vá_para R9;
R9: Se a_zero então vá_para Rx senão vá_para R10;
R10: Faça subtrair_a vá_para R11;
R11: Se a_zero então vá_para Rx senão vá_para R12;
R12: Faça subtrair_a vá_para R13;
R13: Se a_zero então vá_para Rx senão vá_para R14;
R14: Faça subtrair_a vá_para R15;
R15: Se a_zero então vá_para Rx senão vá_para R16;
R16: Faça subtrair_a vá_para R17;
R17: Faça adicionar_b vá_para R13;
```

Solução – Programa iterativo:

Padrão iterativo formal:

descartar os três primeiros pares completos de A;
para cada novo par completo restante, adicionar 1 em B;
se em qualquer etapa faltar unidade para completar o par, encerrar.

Solução – Programa recursivo:

P é R onde

```
R def (se a_zero então √ senão subtrair_a; S)
S def (se a_zero então √ senão subtrair_a; T)
T def (se a_zero então √ senão subtrair_a; U)
U def (se a_zero então √ senão subtrair_a; V)
V def (se a_zero então √ senão subtrair_a; W)
W def (se a_zero então √ senão subtrair_a; Q)
Q def (se a_zero então √ senão subtrair_a; Y)
Y def (se a_zero então √ senão subtrair_a; adicionar_b; Q)
```

Observações: Os três primeiros pares completos são consumidos para representar o “-3”. Somente a partir do quarto par completo passa-se a acumular em B.

30) Equivalência de programas – exercício 30

Enunciado: Obter um programa iterativo e um programa recursivo equivalentes ao monolítico dado.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Faça F vá_para R2;  
R2: Se T1 então vá_para R1 senão vá_para R3;  
R3: Faça G vá_para R4;  
R4: Se T2 então vá_para Rx senão vá_para R1;
```

Solução – Programa iterativo:

```
(F;  
  enquanto T1 faça F;  
  G;  
  até T2 faça  
    (F;  
      enquanto T1 faça F;  
      G))
```

Solução – Programa recursivo:

```
P é R onde  
R def F; (se T1 então R senão G; (se T2 então  $\vee$  senão R))
```

31) Equivalência de programas – exercício 31

Enunciado: Obter um programa iterativo e um programa recursivo equivalentes ao monolítico dado.

Solução – Programa monolítico com instruções rotuladas:

```
R1: Faça S = 0 vá_para R2;  
R2: Faça N = 1 vá_para R3;  
R3: Se N < 100 então vá_para R4 senão vá_para R6;  
R4: Faça S = S + N vá_para R5;  
R5: Faça N = N + 1 vá_para R3;  
R6: Faça escrever(S) vá_para R7;
```

Solução – Programa iterativo:

```
(S = 0;  
N = 1;  
enquanto N < 100 faça  
    (S = S + N; N = N + 1);  
escrever(S))
```

Solução – Programa recursivo:

```
P é S = 0; N = 1; R onde  
R def (se N < 100 então S = S + N; N = N + 1; R senão escrever(S))
```